

VIT BHOPAL

VITyarthi – Flipped Course

PROJECT REPORT

ATTENDANCE SYSTEM

Student Record, Attendance and Result Management

Submitted By

PIHU SINGH

Registration No.: 25BAI11564

Course: VITyarthi – Flipped Course

Submission Date: 18 September 2026

VIT BHOPAL

1. Introduction

The Attendance System is a Java-based academic management project developed as part of the VITYarthi Flipped Course. The system is designed to provide a simple menu-driven way to manage student records, record attendance, enter marks, generate student reports, display all students and view a rank list.

The project follows a modular structure in which the Main program controls the menu and program entry, StudentManager manages student records, Attendance handles attendance-related operations, Result handles marks and ranking, and Student represents the student information such as roll number, name and class.

The project demonstrates object-oriented programming, modular program design, user input handling, validation, and interaction between different classes.

2. Problem Statement

Managing student attendance and academic information manually can be time-consuming and can make it difficult to maintain consistent records. A simple computerized system is required to store student details, record attendance, enter marks, prepare reports and display a rank list through an organized workflow.

The proposed Attendance System addresses this requirement through a menu-driven application that brings the main student-management operations into one system.

3. Objectives

- To create a simple and organized attendance management system.
- To maintain basic student information such as roll number, name and class.
- To provide an option to mark and track attendance.
- To allow marks to be entered for students.
- To provide student reports and a list of all students.
- To generate or display a rank list based on result information.
- To demonstrate modular and object-oriented Java programming.

4. Scope of the Project

The scope of the current project is a menu-driven student academic management application focused on attendance and related student records. The provided design shows the following user-facing operations: Add student, Mark attendance, Enter marks, View student report, View all students, and View rank list.

The supplied design does not specify a database, online login system, cloud deployment, or mobile/web interface. Therefore, these features are outside the documented scope of the current version.

5. Functional Requirements

Module / Function	Description
Student Management	Add and manage student records; student information includes roll number, name and class.
Attendance	Mark and track attendance percentage for students.
Result	Enter marks and provide grades/rank-list information.
Student Report	View a report for a selected student.
View All Students	Display the available student records.
Rank List	Display the rank list based on result information.

Module / Function	Description
Menu & Validation	Display menu options, read the user's choice, validate the choice and show an error for an invalid number.

6. Non-Functional Requirements

Requirement	Project Expectation
Usability	The menu-driven interface should be simple and understandable.
Performance	Menu operations and record-related actions should execute without unnecessary delay for the intended project scale.
Reliability	Valid inputs should produce the corresponding operation consistently.
Error Handling	Invalid menu choices should generate an error message and return the user to the menu.
Maintainability	Separate classes/modules make individual functions easier to modify and test.
Resource Efficiency	The application should use resources appropriate for a small academic project.

7. System Architecture

The supplied class and workflow designs show a simple layered interaction. Main acts as the program entry and menu controller. From Main, the user can access StudentManager, Attendance and Result. StudentManager is associated with Student records. This structure separates responsibilities instead of placing every operation inside one class.

User → Main Menu → StudentManager / Attendance / Result → Student Data / Academic Operations

8. Design Diagrams

8.1 Class Diagram

The class diagram identifies Main, StudentManager, Attendance, Result and Student. Main provides the program entry/menu, StudentManager manages student records, Attendance tracks attendance percentage, Result manages grades and rank-list information, and Student stores roll number, name and class.

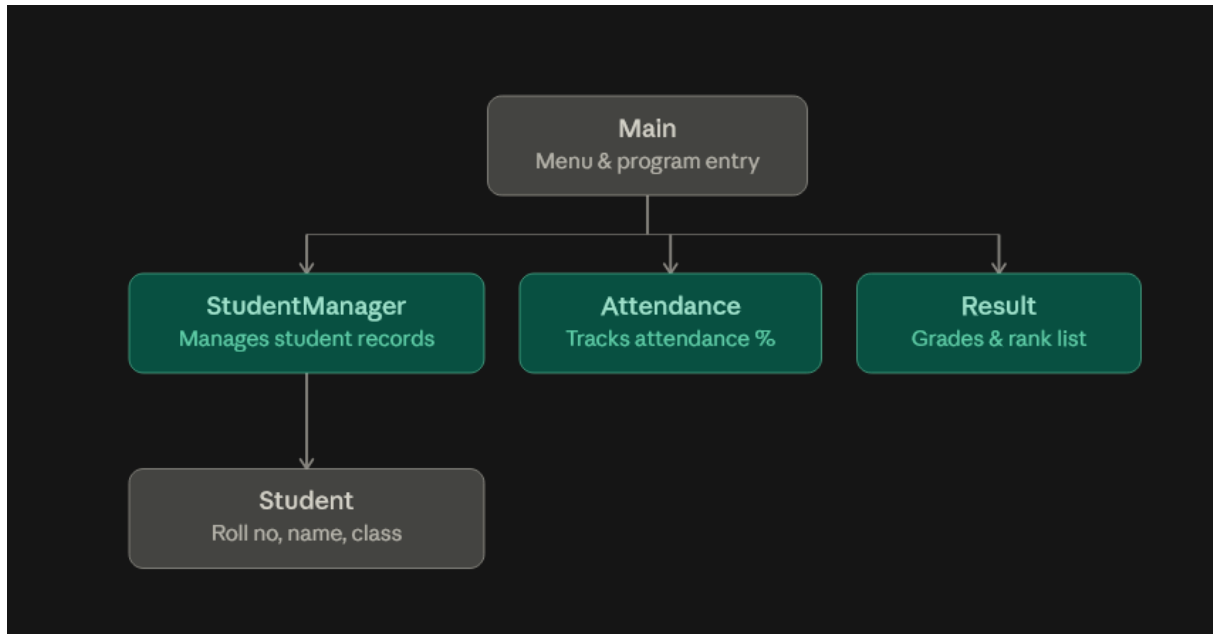


Figure 1. Class Diagram

8.2 Process Flow / Workflow Diagram

The workflow begins with Start, displays the menu, reads the user's choice and checks whether the entered number is valid. An invalid number produces an error message and returns to the menu. A valid choice is checked for Exit; otherwise the selected action is executed and the flow returns to the menu.

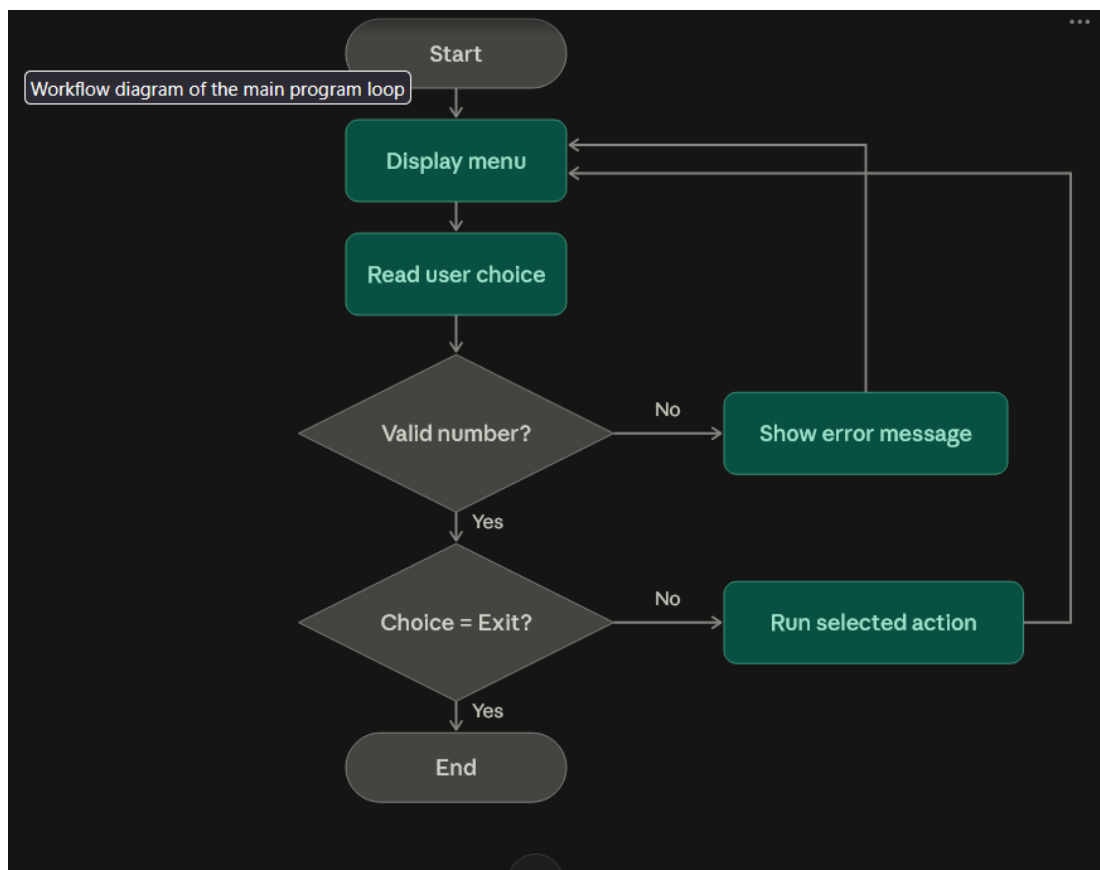


Figure 2. Main Program Workflow

8.3 Use Case Diagram

The use case diagram contains one actor, User. The user can add a student, mark attendance, enter marks, view a student report, view all students and view the rank list.

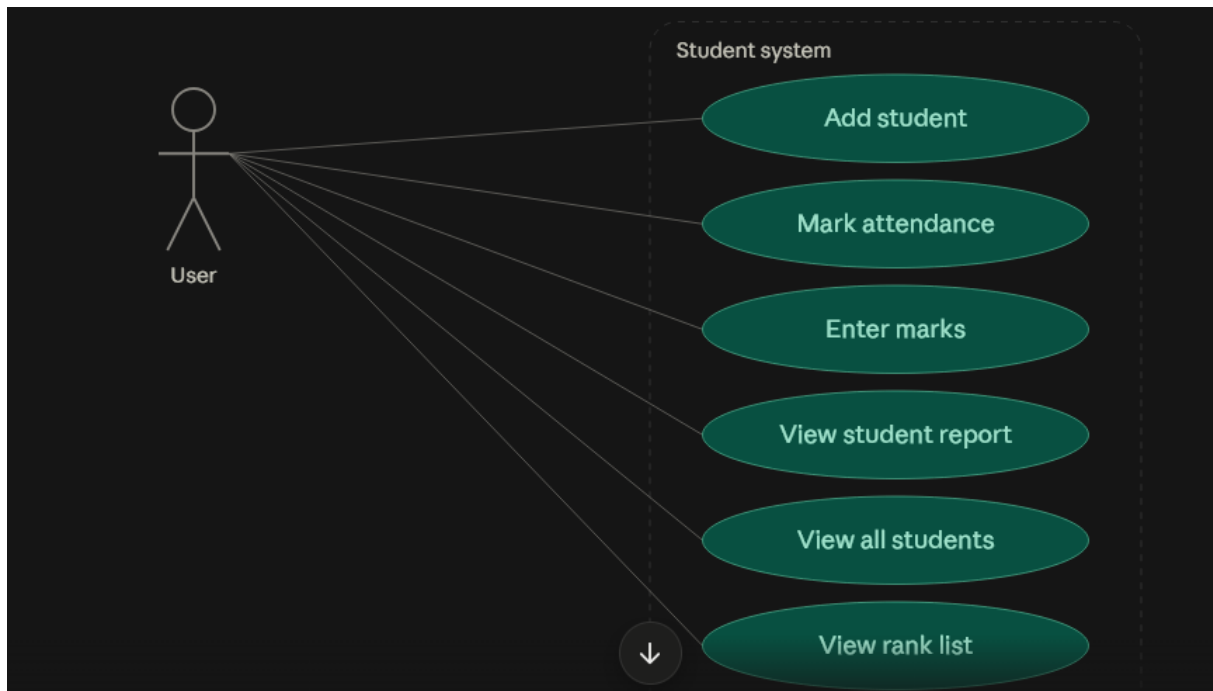


Figure 3. Use Case Diagram

8.4 Sequence Diagram

The sequence diagram illustrates the Enter Marks operation. The User selects Enter marks through Main. Main checks whether the student exists using `studentExists(roll)`. When the student exists, Main sends `addMarks(roll, subj, marks)` to Result. Result returns a confirmation and Main displays the confirmation to the user.

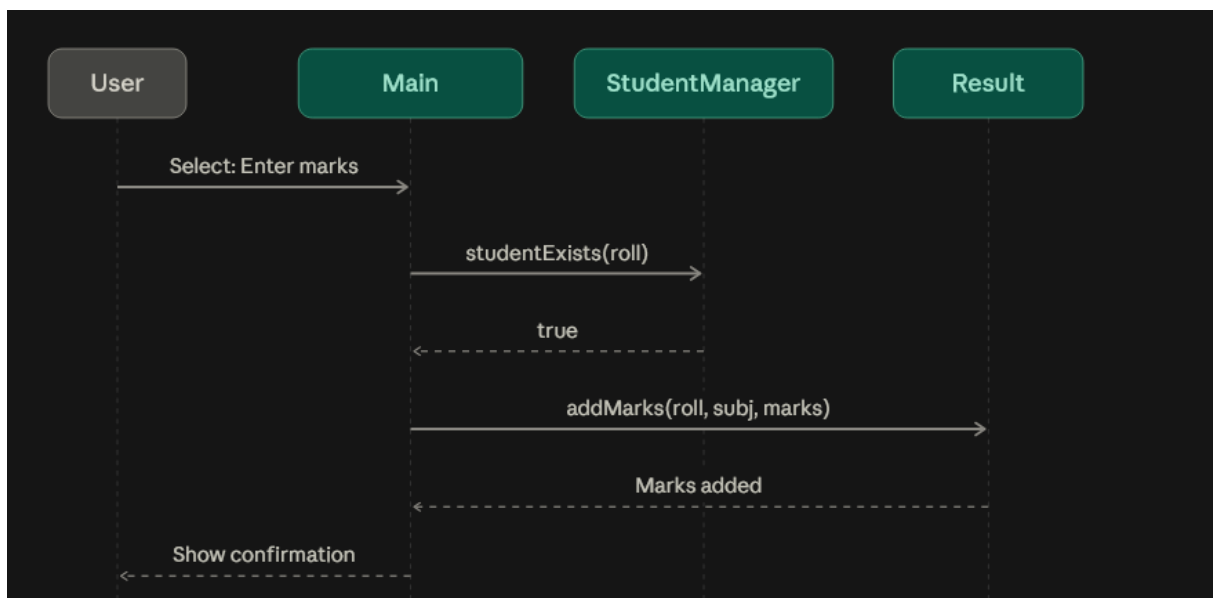


Figure 4. Sequence Diagram – Enter Marks

8.5 Database / Storage Design

No ER diagram or database schema is included in the supplied project material. The current design therefore documents student information and program modules at the class level rather than assuming a particular database technology.

9. Design Decisions and Rationale

- Menu-driven interaction was selected to keep the application straightforward for users.
- Separate classes were used to divide responsibilities among student management, attendance, result processing and the student data model.
- Input validation is included in the main workflow so invalid menu numbers do not directly trigger an operation.
- The design keeps the Main class as the central entry point while delegating specialized tasks to other modules.
- The Student class provides a basic data representation containing roll number, name and class.

10. Implementation Details

The project is organized around five main classes/modules visible in the supplied design: Main, StudentManager, Attendance, Result and Student. These satisfy the project guideline requiring a meaningful set of modules/classes for a coding project.

Class / Module	Primary Responsibility
Main	Program entry, menu display, reading choices, validation and invoking selected actions.
StudentManager	Manages student records and checks whether a student exists.
Student	Represents student details such as roll number, name and class.
Attendance	Handles attendance-related tracking and attendance percentage.
Result	Handles marks, grades and rank-list information.

The workflow also demonstrates method-level communication. For example, the sequence diagram shows `studentExists(roll)` being used before `addMarks(roll, subject, marks)`, which helps prevent marks from being entered for a non-existing student.

11. Screenshots / Results

The supplied design screenshots demonstrate the planned structure and workflow of the Attendance System. The main class/module view shows the relationship between the menu, student management, attendance and result components. The workflow demonstrates validation and menu looping. The use case diagram demonstrates the available user operations. The sequence diagram demonstrates the Enter Marks interaction.

Expected result of the system: the user selects an operation from the menu, the system validates the choice, executes the selected student/attendance/result operation, displays the outcome, and returns to the menu until Exit is selected.

12. Testing Approach

Testing should verify each menu operation separately and then verify the complete workflow. The following test cases are derived directly from the supplied workflow and use-case design.

Test Case	Input / Action	Expected Result
TC-01	Start the program	Main menu is displayed.
TC-02	Enter a valid menu number	Corresponding action is executed.

Test Case	Input / Action	Expected Result
TC-03	Enter an invalid number	Error message is shown and menu is displayed again.
TC-04	Select Exit	Program reaches End.
TC-05	Add student	Student record is created/managed by StudentManager.
TC-06	Mark attendance	Attendance information is recorded/updated.
TC-07	Enter marks for an existing roll number	Marks are added and confirmation is displayed.
TC-08	View student report	Selected student's academic information is displayed.
TC-09	View all students	Available student records are displayed.
TC-10	View rank list	Rank-list information is displayed.

13. Challenges Faced

- Maintaining a clear separation of responsibilities between the different classes.
- Designing a menu loop that continues after each operation and terminates only when Exit is selected.
- Validating menu input and handling invalid numbers without breaking the program flow.
- Ensuring that result operations check the student's existence before adding marks.
- Keeping the workflow, class structure, use cases and sequence of operations consistent.

14. Learnings and Key Takeaways

- Learned how to divide a Java application into multiple classes with focused responsibilities.
- Understood how a main program loop controls repeated user interaction.
- Learned the importance of input validation and error handling.
- Understood how use case, class, workflow and sequence diagrams describe different aspects of the same system.
- Learned how method calls can coordinate operations between modules.
- Gained practical understanding of documenting a software project from requirements through testing.

15. Future Enhancements

- Add persistent database storage so records remain available after the program closes.
- Add authentication and role-based access for authorized users.
- Provide detailed attendance reports and monthly/subject-wise summaries.
- Add graphical reports for attendance and academic performance.
- Add automated low-attendance alerts.
- Provide import/export of student records.
- Develop a web or mobile interface for easier access.
- Add stronger validation for marks, attendance values and duplicate student records.

16. References

- VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines, provided course document.

- Project design diagrams supplied for the Attendance System: Class Diagram, Process Flow Diagram, Use Case Diagram and Sequence Diagram.

Appendix A – Project Structure Summary

Component	Role
Main	Menu & program entry
StudentManager	Manages student records
Attendance	Tracks attendance %
Result	Grades & rank list
Student	Roll no., name, class